

University of Sheffield

How Much Does a Heuristic Buy?

**A* versus Branch and Bound for Least-Effort Walking Routes
on Terrain Maps**



Abdulla Alfalasi

A report submitted for the COM1005 coursework

in the

Department of Computer Science

May 2021

Abstract

A Rambler crosses a height map one cell at a time. Level or downhill steps cost 1, and climbing costs 1 plus the height gained. This report compares branch and bound (uniform-cost search) with A* under five heuristics on the 16×16 course map and three seeded 64×64 synthetic terrains, using 500 random start/goal pairs in total. Four of the heuristics never overestimate the remaining cost: straight-line distance, Manhattan distance, remaining ascent, and Manhattan distance plus remaining ascent. The fifth, absolute height difference, can overestimate it.

All four admissible heuristics returned an optimal route in every run. The combined heuristic did best everywhere. It expanded 25% of the nodes branch and bound expanded on smooth terrain, but 72% on rugged terrain, and a simple informedness measure, $h(\text{start})/C^*$, tracks this gap. The absolute-height heuristic returned a suboptimal route in 22–56% of pairs. On the two noisier 64×64 maps it also expanded more nodes than branch and bound, because it repeatedly reopened closed nodes. These results correct the conclusion of the original 2021 report. That report attributed the gains to Manhattan distance, and it rested on a height heuristic that is not admissible.

Acknowledgements

The search framework and terrain map reader used in this project are the COM1005 base code written by Phil Green (2013) and Heidi Christensen (2021) at the University of Sheffield. The course map `tmc.pgm` was supplied with the same coursework.

Contents

1	Introduction	1
1.1	Aims and Objectives	1
1.2	Overview of the Report	2
2	Background	3
2.1	The Ramblers Problem	3
2.2	Search Algorithms	3
2.2.1	Branch and bound	3
2.2.2	A*	3
2.3	Heuristics	4
3	Methodology	5
3.1	Terrain Maps	5
3.2	Instances	5
3.3	Metrics	5
3.4	Implementation	6
4	Results	7
4.1	Search Effort and Optimality	7
4.2	Why the Savings Shrink on Rugged Terrain	7
4.3	The Cost of an Inadmissible Heuristic	8
4.4	Worked Example	9
4.5	The Original Test Cases	10
5	Discussion	11
5.1	Designing the Heuristic from the Cost Function	11
5.2	Limitations	11
6	Conclusions	12
	Appendices	14
A	Reproducing the Experiments	15
B	Full Results	16

List of Figures

4.1	Mean nodes expanded by A* relative to branch and bound (dashed line). Manhattan+Ascent expands the fewest nodes on every map. The inadmissible absolute-height heuristic does worse than branch and bound on the two noisier 64×64 maps.	8
4.2	Start (green, top right) to goal (yellow, bottom left) on <code>smooth64</code> ; lighter cells are higher. Blue cells were expanded and the red line is the route found. All four routes cost 95.	9

List of Tables

2.1	Heuristics compared. “Admissible” means the heuristic never exceeds the true remaining cost.	4
3.1	Terrain maps used in the experiments	5
4.1	Mean nodes expanded relative to branch and bound (lower is better) and percentage of pairs solved optimally. The last row gives the mean and maximum cost excess of the absolute-height heuristic over the optimum. Branch and bound’s own mean expansions are 148, 2360, 2622 and 2797 for the four maps.	7
4.2	Mean informedness $h(\text{start})/C^*$ of each heuristic. Values above 1 mean the heuristic overestimates.	8
4.3	The three <code>tmc16</code> pairs from the original report: path cost / nodes expanded.	10
B.1	Per-map results for every method.	16

Chapter 1

Introduction

Path planning over terrain is a standard testbed for informed search. The textbook claim is that A* with an admissible heuristic finds the same optimal path as an uninformed method such as branch and bound or Dijkstra's algorithm (Dijkstra 1959, Lawler & Wood 1966), while expanding fewer nodes (Hart et al. 1968, Dechter & Pearl 1985). How many fewer depends on how much of the true cost the heuristic captures, and that in turn depends on the terrain.

1.1 Aims and Objectives

The project asks three questions:

1. How many node expansions does each heuristic save relative to branch and bound, and how does this change with the terrain?
2. Does combining distance and ascent information help, and can the gain be predicted from the heuristic alone?
3. What does an inadmissible heuristic cost in route quality and in search effort?

To answer them, the project:

- implements the Ramblers problem on the COM1005 search framework, with the heuristic selectable per search;
- proves which heuristics are admissible and consistent;
- runs every method on the same 500 seeded start/goal pairs across four maps of increasing roughness;
- re-runs the three test cases from the original report to check that the new code reproduces the old problem.

1.2 Overview of the Report

Chapter 2 defines the problem, the two search algorithms and the five heuristics, and proves that the combined heuristic is admissible and consistent. Chapter 3 describes the maps, the choice of instances, the metrics and the implementation. Chapter 4 reports search effort, optimality and informedness, and re-runs the original test cases. Chapter 5 discusses what the results mean and where they stop, and Chapter 6 concludes. Appendix A explains how to reproduce every number, and Appendix B gives the full per-map results.

Chapter 2

Background

2.1 The Ramblers Problem

A terrain map is a grid of heights $H(r, c) \in \{0, \dots, 255\}$. A state is a cell (r, c) , and the Rambler can move to any of its four neighbours. The cost of stepping from cell u to a neighbouring cell v is

$$c(u, v) = \begin{cases} 1 & \text{if } H(v) \leq H(u), \\ 1 + H(v) - H(u) & \text{if } H(v) > H(u), \end{cases} \quad (2.1)$$

so descending is as cheap as walking on the flat and every metre climbed is paid for. The task is to find a path of least total cost C^* from a start cell to a goal cell.

2.2 Search Algorithms

Both algorithms keep an *open* list of generated nodes and a *closed* list of expanded nodes. A node records its state, its cost so far g , and its parent. When a successor reaches a state already on open or closed by a cheaper route, that node is updated and, if it was closed, moved back to open. Search stops when a goal node is *selected* for expansion, not when it is generated, which is what makes the returned path optimal.

2.2.1 Branch and bound

Branch and bound (Lawler & Wood 1966) always selects the open node with the lowest g . With the duplicate check above it is uniform-cost search, the node-selection form of Dijkstra's algorithm (Dijkstra 1959, Russell & Norvig 2020).

2.2.2 A*

A* selects the open node with the lowest $f = g + h$, where h estimates the remaining cost to the goal (Hart et al. 1968). With $h \equiv 0$ it reduces to branch and bound. This was checked in the code: on every instance tested, A* with $h \equiv 0$ reproduced branch and bound's path cost and node count exactly.

2.3 Heuristics

Let (r, c) be the current cell and (r_g, c_g) the goal. Table 2.1 lists the five heuristics compared.

Table 2.1: *Heuristics compared. “Admissible” means the heuristic never exceeds the true remaining cost.*

Name	Definition	Admissible
Euclidean h_E	$\lfloor \sqrt{(r - r_g)^2 + (c - c_g)^2} \rfloor$	yes
Manhattan h_M	$ r - r_g + c - c_g $	yes
Ascent h_A	$\max(0, H(r_g, c_g) - H(r, c))$	yes
Manhattan+Ascent h_{M+A}	$h_M + h_A$	yes
Absolute height $h_{ \Delta }$	$ H(r_g, c_g) - H(r, c) $	no

Proposition 2.1. h_{M+A} is admissible and consistent, and it dominates h_M , h_E and h_A .

Proof. Write each step cost in (2.1) as $1 + \max(0, \Delta)$, where Δ is the height change of the step. Any path from u to the goal has at least $h_M(u)$ steps. Its positive height changes sum to at least the net rise $H(\text{goal}) - H(u)$, and to at least 0. The path therefore costs at least $h_M(u) + h_A(u) = h_{M+A}(u)$, which proves admissibility. For consistency, take one step from u to v . Then $h_M(u) \leq 1 + h_M(v)$. The inequality $\max(0, a + b) \leq \max(0, a) + \max(0, b)$ gives $h_A(u) \leq \max(0, H(v) - H(u)) + h_A(v)$. Adding the two gives $h_{M+A}(u) \leq c(u, v) + h_{M+A}(v)$. Dominance holds because $h_A \geq 0$, $h_M \geq 0$ and $h_E \leq h_M$. $\square$

$h_{|\Delta|}$ is the “height difference” heuristic of the original version. It also charges for descents, which are cheap. A single downhill step of 100 costs 1, but $h_{|\Delta|}$ estimates it at 100. By standard results, a consistent heuristic that dominates another expands no more nodes, ignoring ties (Pearl 1984, Dechter & Pearl 1985). h_{M+A} is therefore expected to expand the fewest nodes of the admissible heuristics. No such guarantee applies to $h_{|\Delta|}$.

Chapter 3

Methodology

3.1 Terrain Maps

The experiments use the 16×16 map `tmc.pgm` supplied with the coursework, and three 64×64 maps generated as sums of Gaussian hills with per-cell Gaussian noise, rescaled to 0–255 (Table 3.1). The three generated maps range from a few broad hills to many narrow, noisy ones.

Table 3.1: *Terrain maps. Synthetic maps are regenerated from a fixed seed by `TerrainGenerator`.*

Map	Size	Hills	Hill width σ	Noise s.d.	Pairs
<code>tmc16</code> (course map)	16×16	—	—	—	200
<code>smooth64</code>	64×64	8	8–16	0	100
<code>rolling64</code>	64×64	30	3–8	4	100
<code>rugged64</code>	64×64	120	1.5–4	12	100

3.2 Instances

For each map, start/goal pairs were sampled uniformly at random with a fixed seed. Only pairs at least half the map width apart in Manhattan distance were kept, which excludes trivial instances. Every method ran on the same pairs.

3.3 Metrics

Nodes expanded counts every selection from open, including re-expansions of reopened nodes. It is reported relative to branch and bound on the same pair, and the ratio is averaged over pairs.

Optimality compares each path’s cost with branch and bound’s, which is optimal.

Informedness is $h(\text{start})/C^*$, the fraction of the true cost the heuristic sees before the search begins.

Efficiency is the COM1005 measure, path length divided by nodes expanded, kept for continuity with the original report.

Time is wall-clock time per search. Open and closed are linear-scan lists, so time mostly tracks node count, and it varies from run to run.

3.4 Implementation

The search engine (`Search`, `SearchNode`, `SearchState`, `TerrainMap`) is the COM1005 base code by Phil Green and Heidi Christensen. The Ramblers state, the heuristics, the terrain generator and the experiment harness were written for this project. The code is Java 8 compatible, and `run_experiments.sh` regenerates every number and figure in this report (Appendix A). Apart from timing, the results are deterministic.

Chapter 4

Results

4.1 Search Effort and Optimality

Table 4.1 and Figure 4.1 summarise the 500 pairs. All four admissible heuristics returned an optimal route on every pair of every map, as the theory predicts. Their savings differ widely.

Table 4.1: *Mean nodes expanded relative to branch and bound (lower is better) and percentage of pairs solved optimally. The last row gives the mean and maximum cost excess of the absolute-height heuristic over the optimum. Branch and bound’s own mean expansions are 148, 2360, 2622 and 2797 for the four maps.*

Method	tmc16		smooth64		rolling64		rugged64	
	ratio	opt. %	ratio	opt. %	ratio	opt. %	ratio	opt. %
Branch and bound	1.00	100	1.00	100	1.00	100	1.00	100
A* Euclidean	0.83	100	0.57	100	0.74	100	0.85	100
A* Manhattan	0.79	100	0.45	100	0.66	100	0.81	100
A* Ascent	0.79	100	0.82	100	0.81	100	0.91	100
A* Manh.+Ascent	0.56	100	0.25	100	0.48	100	0.72	100
A* Abs. height	0.76	71	0.71	78	1.11	44	1.10	63
Cost excess of A* Abs. height over optimum: mean / max (%)								
	16.9 / 455		2.7 / 51		7.5 / 126		1.9 / 22	

Manhattan+Ascent was the best heuristic on every map. On **smooth64** it expanded a quarter of the nodes branch and bound expanded, and its mean search time fell from 22.4ms to 2.0ms in the recorded run. Distance alone mattered more than ascent alone on the synthetic maps. On the small course map the two did equally well (0.79), and adding them together gave far more than either alone. The ordering $h_{M+A} < h_M < h_E$ in nodes expanded matches the dominance ordering of the three heuristics.

4.2 Why the Savings Shrink on Rugged Terrain

The gain from every heuristic fell as the terrain became rougher: Manhattan+Ascent went from 0.25 on **smooth64** to 0.72 on **rugged64**. Table 4.2 shows why. On smooth terrain

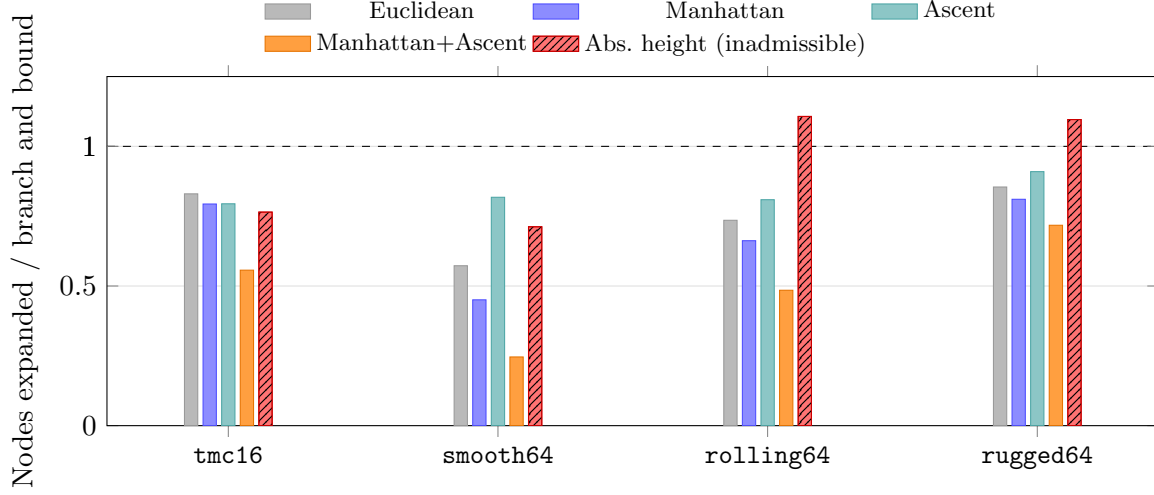


Figure 4.1: Mean nodes expanded by A^* relative to branch and bound (dashed line). Manhattan+Ascent expands the fewest nodes on every map. The inadmissible absolute-height heuristic does worse than branch and bound on the two noisier 64×64 maps.

the optimal route is close to a straight walk plus the unavoidable climb, so h_{M+A} sees 79% of the true cost from the start. On rugged terrain the optimal route pays for many small climbs and detours that no local estimate can see, and h_{M+A} sees only 27%. Across the three synthetic maps, which share a size, lower informedness goes with a higher expansion ratio for every admissible heuristic except Ascent, whose informedness is low everywhere. Unlike the expansion ratio, informedness can be measured from a sample of solved instances without comparing search runs.

Table 4.2: Mean informedness $h(\text{start})/C^*$ of each heuristic. Values above 1 mean the heuristic overestimates.

Heuristic	tmc16	smooth64	rolling64	rugged64
Euclidean	0.20	0.49	0.27	0.15
Manhattan	0.27	0.64	0.35	0.20
Ascent	0.24	0.15	0.13	0.07
Manhattan+Ascent	0.51	0.79	0.48	0.27
Abs. height	1.72	0.63	0.44	0.16

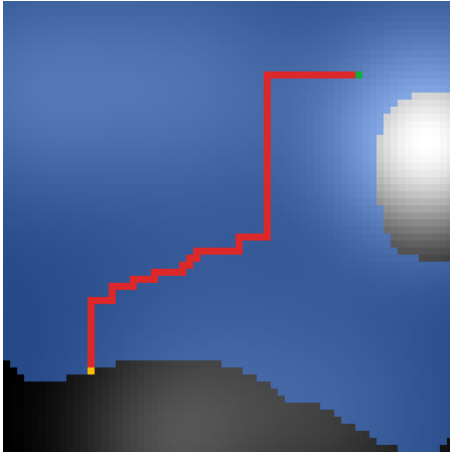
4.3 The Cost of an Inadmissible Heuristic

The absolute-height heuristic returned a suboptimal route on 22–56% of pairs. The worst route on the course map cost 61 against an optimum of 11, 5.5 times as much. It was also not reliably faster. On `rolling64` and `rugged64` it expanded about 10% *more* nodes than branch and bound, and on one `rolling64` pair it expanded 20,047 nodes on a 4,096-cell map.

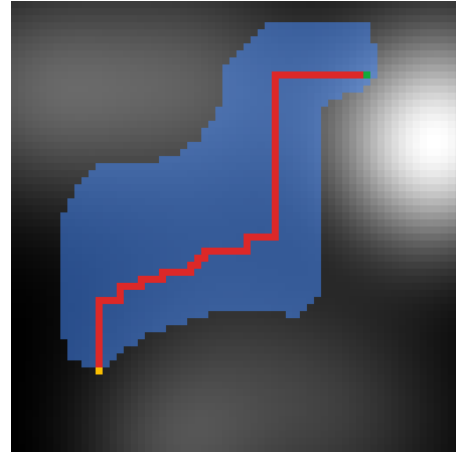
An inconsistent heuristic can close a node by an expensive route and then find a cheaper one. The node is then reopened and expanded again, and those re-expansions add up. On the course map it has a mean informedness of 1.72, so it usually overestimates, and its apparent savings there come largely from that overestimation. It prunes aggressively by ignoring routes that are actually cheap.

4.4 Worked Example

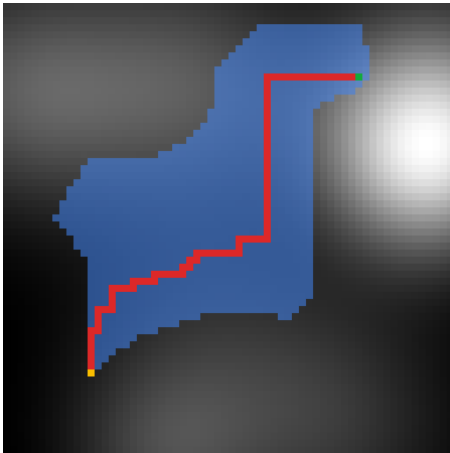
Figure 4.2 shows one pair on `smooth64`. All four methods returned routes of cost 95. Branch and bound expanded 3322 nodes, most of the map. Manhattan expanded 1221 and Manhattan+Ascent 1148, both confined to a corridor around the route. The absolute-height heuristic still found the optimum here but expanded 4662 nodes, more than the map contains, because of re-expansions.



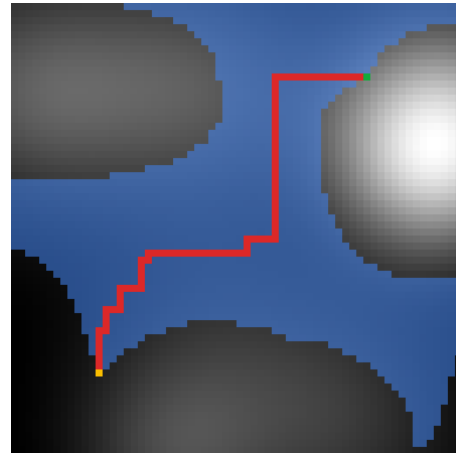
(a) Branch and bound: 3322 expanded



(b) A* Manhattan: 1221 expanded



(c) A* Manhattan+Ascent: 1148 expanded



(d) A* Abs. height: 4662 expanded

Figure 4.2: Start (green, top right) to goal (yellow, bottom left) on `smooth64`; lighter cells are higher. Blue cells were expanded and the red line is the route found. All four routes cost 95.

4.5 The Original Test Cases

Table 4.3 re-runs the three pairs from the original report. Branch and bound’s efficiencies (0.0618, 0.0649, 0.2105) match the original figures (0.0617, 0.0649, 0.210), which confirms that the rewritten state class reproduces the original problem. The column labelled “No. visited” in the original report was the path length, not the number of nodes visited. With the corrected heuristics, the case for Manhattan alone is weak on this map: it saves at most 17% of expansions. The absolute-height heuristic returns suboptimal routes on two of the three pairs, with costs 150 against 144 and 69 against 43.

Table 4.3: *The three tmc16 pairs from the original report: path cost / nodes expanded.*

Method	$(2, 4) \rightarrow (4, 8)$	$(2, 4) \rightarrow (8, 12)$	$(2, 4) \rightarrow (10, 15)$
Branch and bound	100 / 178	144 / 231	43 / 114
A* Euclidean	100 / 174	144 / 229	43 / 98
A* Manhattan	100 / 173	144 / 229	43 / 95
A* Ascent	100 / 43	144 / 159	43 / 114
A* Manhattan+Ascent	100 / 42	144 / 118	43 / 95
A* Abs. height	100 / 46	150 / 174	69 / 158

Chapter 5

Discussion

5.1 Designing the Heuristic from the Cost Function

The practical lesson is to build the heuristic from the cost function. Each step in this problem pays for distance and for climbing, and the heuristic that bounds both terms beat the two that bound only one. Informedness is a cheap diagnostic: it can be computed from a sample of solved instances before choosing a heuristic, and on the synthetic maps it tracked the terrain effect.

The absolute-height heuristic shows the opposite case. It looks reasonable because height matters in this problem, but it prices descents as if they were climbs. That makes it overestimate, which costs optimality, and it makes it inconsistent, which on the larger maps cost more search than using no heuristic at all.

5.2 Limitations

- The synthetic maps come from one generator with one seed per roughness level, so the terrain effect is shown on three maps, not a distribution of them.
- Movement is 4-connected. Diagonal moves would change both the costs and which heuristics are admissible.
- Open and closed are linear lists, as in the base code, so timings would shrink with a priority queue and a hash set. The node-count comparisons would not change.
- Ties in f are broken by list order, which can shift node counts slightly between heuristics with equal f values.

Chapter 6

Conclusions

On least-effort terrain routing, A* with any admissible heuristic returned the same optimal routes as branch and bound, and it expanded between 25% and 91% as many nodes, depending on the heuristic and the terrain. A heuristic that combines Manhattan distance with remaining ascent is admissible and consistent, and it was the best choice on every map. Its advantage shrinks on rugged terrain, as its informedness predicts. The absolute-height heuristic from the original study is not admissible. It returned suboptimal routes on up to 56% of pairs, and on the two noisier 64×64 maps it searched more than branch and bound did.

Natural extensions are 8-connected movement, more terrain samples per roughness level, and a priority-queue implementation so that timing comparisons become meaningful alongside node counts.

Bibliography

- Dechter, R. & Pearl, J. (1985), ‘Generalized best-first search strategies and the optimality of A*’, *Journal of the ACM* **32**(3), 505–536.
- Dijkstra, E. W. (1959), ‘A note on two problems in connexion with graphs’, *Numerische Mathematik* **1**, 269–271.
- Hart, P. E., Nilsson, N. J. & Raphael, B. (1968), ‘A formal basis for the heuristic determination of minimum cost paths’, *IEEE Transactions on Systems Science and Cybernetics* **4**(2), 100–107.
- Lawler, E. L. & Wood, D. E. (1966), ‘Branch-and-bound methods: A survey’, *Operations Research* **14**(4), 699–719.
- Pearl, J. (1984), *Heuristics: Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley.
- Russell, S. & Norvig, P. (2020), *Artificial Intelligence: A Modern Approach*, 4th edn, Pearson.

Appendices

Appendix A

Reproducing the Experiments

The code is at <https://github.com/aalbudoor/astar-vs-branch-and-bound>. With a JDK (version 8 or newer) installed, one command regenerates the synthetic maps, every result file and every figure:

```
./run_experiments.sh
```

It compiles `src/` into `out/` and runs `Experiment`, which writes:

`results/runs.csv` one row per map, pair and method;

`results/summary.csv` per-map, per-method aggregates;

`results/original_cases.csv` the three test cases from the original report;

`results/figures/` the images in Figure 4.2.

The full run takes about 20 seconds. Apart from timings, the output is identical from run to run.

A single instance can be solved from the command line:

```
javac -d out src/*.java
```

```
java -cp out RunRamblers maps/tmc.pgm 0 0 15 15 manhattan_ascent path.png
```

The method is one of `bb`, `zero`, `euclidean`, `manhattan`, `ascent`, `manhattan_ascent` or `abs_height`.

Appendix B

Full Results

Table B.1 gives the mean and median nodes expanded, the COM1005 efficiency measure, and the mean wall-clock time per search for every map and method. Times come from one run on a laptop and vary between runs; the other columns do not.

Table B.1: *Per-map results for every method.*

Map	Method	Nodes expanded		Efficiency	Time (ms)
		mean	median		
tmc16	Branch and bound	148	146	0.142	0.1
	A* Euclidean	130	128	0.195	0.1
	A* Manhattan	126	118	0.215	0.1
	A* Ascent	110	106	0.184	0.1
	A* Manhattan+Ascent	80	73	0.297	0.1
	A* Abs. height	105	103	0.211	0.1
smooth64	Branch and bound	2360	2392	0.026	22.4
	A* Euclidean	1484	1222	0.059	11.3
	A* Manhattan	1209	877	0.094	8.6
	A* Ascent	1863	1738	0.032	14.4
	A* Manhattan+Ascent	595	482	0.129	2.0
	A* Abs. height	1736	1594	0.056	12.3
rolling64	Branch and bound	2622	2685	0.024	26.9
	A* Euclidean	1998	1907	0.035	17.3
	A* Manhattan	1814	1716	0.040	14.8
	A* Ascent	2073	2031	0.031	17.9
	A* Manhattan+Ascent	1277	1142	0.057	7.9
	A* Abs. height	2981	2175	0.032	25.2
rugged64	Branch and bound	2797	3024	0.024	30.2
	A* Euclidean	2433	2488	0.028	23.8
	A* Manhattan	2322	2268	0.030	22.1
	A* Ascent	2528	2580	0.026	25.1
	A* Manhattan+Ascent	2035	1878	0.034	17.4
	A* Abs. height	3062	2852	0.023	29.6